

AgroSense Project Diagnosis and Workflow Handbook

Date: 2026-04-04

Workspace: cropd

1) Executive Summary

AgroSense is a full-stack agriculture assistant with:

- Django + Django REST backend for APIs.
- Static HTML/CSS/JavaScript frontend served separately.
- TensorFlow-based crop disease image detection.
- NLP chatbot with intent classification + knowledge retrieval.
- OTP email authentication and JWT authorization.
- MongoDB for operational/user data and SQLite for Django app persistence.

Current maturity level:

- Good for local demos and feature experimentation.
- Not yet production-ready due to security defaults, lack of tests, and a blocking syntax issue in training code.

Overall technical health score: 5.5/10.

2) What Technologies Are Used and Why

Backend

- Django 5.x: mature Python web framework with routing, middleware, and admin ecosystem.
- Django REST Framework: API development with serializers and class-based views.
- PyMongo: direct access to MongoDB collections for flexible document data.
- python-dotenv: environment variable loading from .env for secrets/config.

Why this stack:

- Fast API development and easy integration with Python ML code.
- Django middleware enables centralized JWT extraction.
- MongoDB suits user activity/history records with flexible schema.

Machine Learning

- TensorFlow / Keras: image classification model loading and inference.
- MobileNetV2 preprocessing utilities: standardized input pipeline for image normalization.
- scikit-learn (TF-IDF + Logistic Regression): lightweight intent classifier for chatbot.

Why this stack:

- TensorFlow handles production-grade image models.
- TF-IDF + Logistic Regression is fast and reliable for intent routing.
- Mix of deterministic intent + retrieval gives explainable chatbot behavior.

Frontend

- Plain HTML/CSS/JavaScript (no framework): low setup overhead, easy local hosting.
- i18n batching via backend translation API.

Why this stack:

- Quick prototyping and direct control of UI behavior.
- Easy to serve with Python http.server in development.

Runtime and Tooling

- PowerShell startup scripts for local orchestration.
- Separate backend and frontend ports (8000 / 3000).

Why this setup:

- Clear separation of concerns and simple debugging during development.

3) Project Structure and Responsibilities

- agrosense_backend/agrosense_backend/settings.py: global Django config, middleware, CORS, email config.
- agrosense_backend/agrosense_backend/urls.py: root API routing (/api/...).
- agrosense_backend/api/urls.py: feature-level endpoint registration.
- agrosense_backend/api/auth/: OTP send/verify and JWT utilities.
- agrosense_backend/api/detect/: image upload, disease prediction API, detection history.
- agrosense_backend/api/ml_model.py: model initialization, inference pipeline, class map loading.
- agrosense_backend/api/chatbot/: chat orchestration, model training, knowledge retrieval, integrations.
- agrosense_backend/api/market/: market price fetch and history.
- agrosense_backend/api/schemes/: government schemes listing + tracking.
- agrosense_backend/api/views/: translation, health, feedback, activity history.
- agrosense_backend/api/mongo_client.py: MongoDB connection singleton and availability checks.
- frontend_new/: static frontend pages and scripts.
- start_backend.ps1, start_frontend.ps1, start_dev.ps1: local startup automation.

4) API Surface and Functional Workflow

Authentication Workflow (OTP + JWT)

1. Frontend sends email to /api/auth/send-otp/.
2. Backend generates OTP and emails it.
3. OTP record is stored in MongoDB with expiry.
4. Frontend submits OTP to /api/auth/verify-otp/.
5. Backend verifies OTP, upserts user record, returns JWT.
6. Frontend stores token and sends Authorization: Bearer <token>.

Why this approach:

- Passwordless onboarding reduces friction.
- JWT supports stateless auth across API endpoints.

Disease Detection Workflow

1. User uploads plant image from frontend.
2. /api/detect/ validates request and saves temporary image.
3. ml_model.py preprocesses image and runs prediction.
4. Top result + confidence + top-3 classes are produced.
5. disease_info.py enriches output with cure/reference info.
6. For logged-in users, record is asynchronously stored in MongoDB.

7. Temporary image is cleaned up after response.

Why this approach:

- Async persistence keeps API responsive.
- Model timeout and guarded inference reduce hangs.

Chatbot Workflow

1. Frontend sends message to /api/chat/message/.
2. chatbot.py predicts intent via TF-IDF + Logistic Regression model.
3. Depending on intent, it routes to:
 - market service
 - weather service
 - schemes data
 - disease knowledge retrieval
4. If confidence is low, fallback logic/knowledge search is used.
5. Authenticated interactions can be logged for history.

Why this approach:

- Intent routing provides predictable responses.
- Retrieval fallback improves coverage beyond templates.

Schemes and Market Workflows

- /api/schemes/ filters by crop/state/search and returns scheme cards.
- /api/market-price/ queries external API with fallback mock data.

Why this approach:

- Keeps farmer-facing utility features available even with partial outage.

Translation Workflow

- Frontend batches UI strings.
- /api/translate-ui/ translates and returns mapped output.
- Frontend caches translated strings in localStorage.

Why this approach:

- Reduced API calls and faster repeated page interactions.

5) Data Layer: What Is Stored Where and Why

MongoDB (operational/application data)

Used for:

- OTP lifecycle records.
- User records and profile metadata.
- Disease prediction history.
- Market query history.
- User activity timeline.
- Feedback records.

Why MongoDB:

- Flexible schema for evolving product events and logs.
- Easy append-heavy writes for activity streams.

SQLite (Django persistence)

Used for:

- Core Django app data/models and migration-managed entities.

Why SQLite currently:

- Zero-setup local development.
- Simple baseline storage for early-stage development.

Model and Knowledge Assets

- Keras model files + class labels for image detection.
- JSONL/CSV corpora for chatbot knowledge and training.

Why file assets:

- Direct model loading with minimal serving complexity.

6) Local Development Workflow

Start commands

- Backend: ./start_backend.ps1
- Frontend: ./start_frontend.ps1
- Both: ./start_dev.ps1

Ports

- Backend API: http://127.0.0.1:8000
- Frontend app: http://127.0.0.1:3000

Typical developer loop

1. Start backend and frontend.
2. Open frontend and perform OTP login or guest mode.
3. Test disease detection, chat, schemes, market, history.
4. Verify backend logs for errors/warnings.
5. Iterate feature files in api/ and frontend_new/.

7) Diagnostic Results (Current State)

Automated checks run

- Django system check: PASSED.
- Frontend JavaScript syntax check: PASSED.
- Python compile-all on api/: FAILED due to syntax error.
- Django tests: ran 0 tests (no effective test suite).

High severity findings

1. Syntax error in chatbot training script:
 - File: agrosense_backend/api/chatbot/train_chatbot.py
 - Problem: typo token "pimport random".
 - Impact: training script cannot run.
2. Security defaults unsafe for production:
 - settings.py has DEBUG=True, ALLOWED_HOSTS=["*"], CORS_ALLOW_ALL_ORIGINS=True.
 - SECRET_KEY is hardcoded in source.

3. No real test coverage:
- api/tests.py is placeholder only.
 - manage.py test returns 0 tests.

Medium severity findings

1. Package hygiene issues:
 - Extra typo files `_init_.py` inside auth and detect directories.
2. Data quality issues in `disease_info.py`:
 - Duplicated malformed URL for pea leaf miner reference.
 - Inconsistent key with embedded extra space (`Pea_ DOWNY_MILDEW_LEAF`).
3. Portability issue:
 - `knowledge_engine.py` contains absolute Windows path hardcoding.
4. Broad exception handling:
 - Multiple bare except/except Exception blocks in chatbot and auth code reduce debuggability.

Low severity findings

- Missing README/user-facing architecture document.
- Inconsistent formatting and naming conventions across some modules.

8) Strengths

- Clear feature modularization (auth, detect, chatbot, schemes, market, views).
- Practical offline/degraded behavior when MongoDB/external services are unavailable.
- Good user-oriented functionality breadth (detection + advisory + market + schemes + i18n).
- Startup scripts make local onboarding easier.
- ML inference pipeline includes timeout protections and logging.

9) Risks if Deployed As-Is

- Unauthorized access risk due to permissive host/CORS and debug mode.
- Operational blind spots due to zero test coverage.
- Training/retraining pipeline blocked by syntax error.
- Environment portability issues from hardcoded absolute paths.
- Data consistency drift due to hardcoded disease dictionary quality issues.

10) Recommended Improvement Roadmap

Immediate (1-2 days)

1. Fix training syntax error and remove duplicate/redundant training block.
2. Move `SECRET_KEY` and all sensitive runtime config to `.env` only.
3. Set `DEBUG=False` and restrict `ALLOWED_HOSTS/CORS` by environment.
4. Remove/rename stray `_init_.py` files.
5. Fix `disease_info` key and malformed URL entries.

Short term (1 week)

1. Add API smoke tests for auth, detect, chat, schemes, market.

2. Add unit tests for JWT utils and Mongo availability fallback.
3. Replace hardcoded Windows paths with BASE_DIR-relative paths.
4. Add centralized error response shape and typed logging contexts.

Mid term (2-4 weeks)

1. Add CI pipeline: lint + test + compile checks.
2. Externalize model serving strategy and artifact versioning.
3. Add role-based access and rate limiting for OTP verify.
4. Add production deployment profile (WSGI/ASGI server, env-based config).

11) Everything You Need to Know (Quick Reference)

- Main backend entry: agrosense_backend/manage.py
- Main API router: agrosense_backend/api/urls.py
- Main frontend script: frontend_new/home.js
- JWT middleware: agrosense_backend/api/middleware/auth_middleware.py
- Mongo connector: agrosense_backend/api/mongo_client.py
- Detection model logic: agrosense_backend/api/ml_model.py
- Chatbot orchestrator: agrosense_backend/api/chatbot/chatbot.py
- Startup scripts: start_backend.ps1, start_frontend.ps1, start_dev.ps1

If you continue this project, the first critical step is to address security defaults and introduce minimum automated tests before adding new features.

End of handbook.